

Amaliy mashg'ulot № 28

Mavzu: Metodning massivli parametrlari va params kalit so'zi

Ishning maqsadi: Talabalarga rekursiv metodlar haqida tushunchalar berish. Rekursiv metodlardan foydalanib masalalar yechish yo'llarini o'rgatish.

Kerakli asbob-uskunalar va jihozlar: fan bo'yicha O'UM, dasturlash bo'yicha topshiriqlar to'plami, kompyuter, videoproektor.

Masalaning qo'yilishi: Rekursiv metodlardan foydalanib masalalar yechish.

Qisqacha nazariy ma'lumotlar.

Murakkab toifadagi dasturlar tuzishda, foydalanuvchi tomonidan aniqlanadigan funksiyalar e'lon qilinishi kerak. Obyetga yo'naltirilgan dasturlashda funksiyalar metod sifatida qaraladi.

Hozirgacha ko'rib chiqilgan obyektlarda metodlar yozilmagan, faqat bitta standart Main() metodidan foydalanildi:

```
public static void Main()
{
    Console.WriteLine("Hello World!");
}
```

Metod nomidan keyin qavs ichida (parametr yoki parametrsiz), nomdan oldin esa metodning turi ko'rsatiladi. Bizning holatda, metod qaytaradigan qiymat turi **public static void** (ochiq, statik, bo'sh). Hozircha faqat **void** kalit so'zini ko'rib chiqildi, ya'ni qaytariladigan qiymat yo'q. Agar siz Delphi dasturlash tili bilan tanish bo'lsangiz, **void** xususiyatini protseduraga havola sifatida ko'rishingiz mumkin. C# va C++ da protsedura tushunchasi mavjud emas, faqat funksiyalar mavjud. Lekin agar **void** funksiya turi ko'rsatilgan bo'lsa, u holda funksiya *qiymat qaytarmaydi*, ya'ni protsedura bilan bir xil ishlaydi.

Parametrlarni qavs ichidagi metod nomidan keyin, qavslarning ichiga kiritish mumkin.

```
Metod_nomi(parametrlar)
{
    //Metod tanasi
}
```

Bu yerda, qavslar ichidagi parametrlar metodda ishlatilishi kerak bo'lgan argumentlar. Figurali qavslar (“{ }”) - bu metodni bajaradigan kod (harakat). Agar metod tanasida bitta amal bajarilib, o'tkazib yuborilishi mumkin bo'lsa ham, metodda hech qanday harakat bo'lmasa ham, metodning boshlanishi va oxirini belgilovchi “{ }” belgilarini qo'yish talab qilinadi.

Bu satr boshqa sinf metodidan chaqirilsa uning sintaksisi quyidagicha bo'ladi:

```
Obyekt_nomi.Metod_nomi(Parametrlar);
```

Bu yerda, Obyekt_nomi boshqa sinfda joylashganligi uchun yaratiladigan obyekt. Agar metod uni chaqiradigan obyekt (sinf) ichida bo'lsa, chaqirish quyidagicha qisqartirish mumkin:

```
Metod_nomi(Parametrlar);
```

Agar xossa yoki sinf o'zgaruvchisiga kirish kerak bo'lsa, u holda xossa yoki o'zgaruvchining nomi ham to'liq yozilishi kerak:

```
Obyekt_nomi.Xossa_nomi;
```

Ammo, agar bir sinfga tegishli metoddan sinf xossasiga kirish talab qilinsa, u holda obyekt o'zgaruvchisini ko'rsatmasdan faqat xossa yoki o'zgaruvchining nomini yozish kerak.

Main() metodi asosiy metod bo'lib, dastur o'z ishini aynan undan boshlaydi. Bu shuni anglatadiki, kamida bitta sinfda shunday nomli metod bo'lishi kerak, aks holda dasturni qayerdan boshlanishi aniq bo'lmaydi. Main() metodi faylning boshida biror joyda bo'lishi shart emas va uni istalgan joyda joylashtirish mumkin, chunki ijro fayl boshidan oxirigacha emas, balki Main() dan boshlab, keyin esa qolgan amallar bajariladi. Agar sizda C++ yoki Delphi kabi tillarda dasturlash tajribangiz bo'lsa, C# da metodlarni realizatsiya qilish bir vaqtning o'zida sodir bo'ladi. Dastur kodi va realizatsiya qilish bir faylda bo'lganligi sababli, kod yanada ixcham va mobilroq bo'ladi. Bu .NET texnologiyasining asosiy xususiyatlaridan biridir.

Metodlarning tavsifi

Endi eng oddiy metodlarni yaratish va ulardan foydalanishni o'rganamiz. Umuman olganda, metod e'loni quyidagicha ko'rinadi:

modifikator **qiymat_turi** **Funksiya_nomi**(**parametr_turi** parametrlar, <vergullar bilan ajratiladi>)

```
{  
    //Kod  
    return Qiymat;  
}
```

Kirish modifikatorlari public, protected va private bo'lishi mumkin, ular yordamida bu metod tashqi sinflar yoki vorislik uchun mavjud yoki yo'qligini aniqlash mumkin, bu haqda keyingi boblarda batafsil ma'lumot beriladi. Agar metod qiymatni qaytarmasa, void kalit so'zini ko'rsatish kerak. Agar metod parametrlarga ega bo'lmasa, qavs ichida hech narsa ko'rsatilishi shart emas.

Parametrli metodlarga misollar:

1. **public static void metod1**(**int** a)
 {
 // metod tanasi
 }
2. **public static void metod2**(**int** a, **double** b)
 {
 // metod tanasi
 }
3. **public static int metod3**(**int** a, **double** b)
 {
 // metod tanasi
 return qiymat;
 }
4. **public static string metod4**(**string** a, **int** b)
 {
 // metod tanasi
 return qiymat;
 }

Figurali qavslarda biz metod kodini yozishimiz mumkin va agar metod qiymatni qaytarishi kerak bo'lsa, u return kalit so'zidan keyin ko'rsatilishi kerak. Bu operator metod kodining istalgan joyida, hatto eng boshida ham bo'lishi mumkin, lekin u metodni to'xtatib, qiymat qaytaradi. Agar metod qiymat qaytarmasa yoki to'g'rirog'i void bo'lsa, u holda return ixtiyoriy, faqat metodni bekor qilish uchun ishlatish mumkin.

Quyidagi dastur matni beradigan metod yaratamiz:

```
using System;  
namespace Funksiya  
{  
    public class Shed  
    {  
        public int GetSize(int width, int height, int length)  
        {  
            int size = width * height * length;  
            return size;  
        }  
    }  
}
```

Bu yerda, **GetSize()** nomli metod e'lon qilingan. U ochiq (public) deb e'lon qilinadi, shuning uchun o'lchamni qaytaradigan metod public deb e'lon qilinadi.

Biz jismning o'lchamlarini butun sonlar sifatida belgilab qo'ydik, ularni haqiqiy son sifatida berishimiz ham mumkin. O'lchamlar butun son bo'lganligi sababli, jismning hajmi ham butun son bo'ladi, chunki *hajm eni, bo'yi va balandlikning ko'paytmasidir* va butun sonlarni ko'paytirish natijasi ham, har doim butun son bo'ladi. Dasturlashda toshib ketish (maksimal son hajmidan oshib ketish) kabi tushuncha ham mavjud, ammo biz buni hozircha ko'rib chiqmaymiz.

GetSize() metodi sinf o'zgaruvchilari ko'rinishida hisoblash uchun zarur bo'lgan barcha narsaga kirish huquqiga ega.

Metod ichida birinchi qatorda *size* o'zgaruvchisi e'lon qilinadi va unga jismning barcha uchta o'lchamini ko'paytirish natijasi qiymatlanadi. Ikkinchi qatorda biz return kalit so'zidan foydalanib, hisoblash natijasini qaytaramiz. Hammasi bir qatorda bajarilishi ham mumkin, ya'ni:

```
public int width;  
public int height;  
public int length;  
public int GetSize()  
{  
    return width * height * length;  
}
```

Bu yerda, return kalit so'zidan so'ng, metodning uchta o'zgaruvchini metod parametridan olib, sinf maydoni sifatida kiritib, ko'paytirish natijasini qaytarishi kerakligini ifoda shaklida ko'rsatdik.

Topshiriqlar.

1. $n!$ ni hisoblaydigan haqiqiy tipli **Fact(n)** funksiyasi tasvirlansin. Berilgan 5 ta son uchun tasvirlangan funksiyadan foydalanib faktoriallar hisoblansin.
2. Ikkilangan faktorialni hisoblovchi $(n \neq n \cdot (n-2) \cdot (n-4) \cdot \dots)$ rekursiv haqiqiy tipli **Fact2(n)** funksiyasi tasvirlansin. (Bu erda $n > 0$. Agar oxirgi ko'paytiruvchi juft bo'lsa $n=2$, toq bo'lsa $n=1$ deb olinsin.) Berilgan 5 ta son uchun bu funksiya yordamida ikkilangan faktoriallar hisoblansin.
3. x ning n -darajasini hisoblaydigan, ($x^0=1$, n juft bo'lganda $x^n = \left(x^{\frac{n}{2}}\right)^2$ ($n > 0$), n toq bo'lganda

$x^n = x \cdot x^{n-1}$ $n > 0$, $n < 0$ bo'lganda $x^n = \frac{1}{x^{-n}}$ rekursiv haqiqiy tipli **PowerN(x,n)** funksiyasi

tasvirlansin. (x haqiqiy tipga tegishli ($x \neq 0$), n -butun tipga tegishli). Bu funksiya yordamida berilgan x uchun n ning 5 ta turli qiymatlaridagi x ning darajalari hisoblansin.

4. n -tartibli Fibonachchi sonlarini hisoblaydigan (n -butun son) rekursiv butun tipli **Fib1(n)** funksiyasi tasvirlansin. Bu funksiya yordamida turli 5 ta butun son uchun Fibonachchi sonlari hisoblansin, har bir n uchun **Fib1** funksiyasini chaqirishlar soni ham aniqlansin.
5. n ta elementli Fibonachchi sonlari ketma-ketligini hisoblaydigan butun tipli **Fib2** protsedurasi tasvirlangan. Bunda $n \leq 20$. yordamchi massiv tuzilsin va unga hisoblangan fibonachi sonlari yozilib, **Fib2** funksiyasi bajarilayotganda murojaat etilsin. (Bu yerda n yigirmadan oshmaydigan butun son). **Fib2** funksiyasi yordamida $n=5$ ta har xil qiymatlari uchun Fibonachchi sonlari hisoblansin va hisoblashda **Fib2** funksiyasini chaqirishlar soni **Fib1** funksiyasini chaqirishlar soni bilan taqqoslang.
1. 6. $\Delta(n,k)$ ($c(n,0)=c(n,n)=1$, $c(n,k)=c(n-1,k)+c(n-1,k-1)$ $0 < k < n$ bo'lsa) ifodani hisoblovchi rekursiv butun tipli **Combin1(n,k)** funksiyasi tasvirlansin. $n > 0$, $0 \leq k \leq n$ bo'lgan holda, berilgan n sonida k ning turli 3 ta qiymati uchun $c(n,k)$ soni hisoblansin va $c(n,k)$ ni hisoblashda **Combin1** rekursiv funksiyani chaqiruvlar soni ham aniqlansin.
6. $c(n,k)$ ($c(n,0)=c(n,n)=1$, $c(n,k)=c(n-1,k)+c(n-1,k-1)$ $0 < k < n$ bo'lsa) ifodani hisoblaydigan parametrlari butun tipga tegishli rekursiv butun tipli **Combin2(n,k)** funksiya tasvirlansin. $c(n,k)$ ning hisoblangan qiymatlarini o'zida saqlovchi ikki o'lchovli massiv tuzilsin. **Combin2** da bu massivdan foydalanilsin. Ushbu funksiya yordamida berilgan n ning qiymati uchun k ning 5 ta har xil qiymatlaridagi $c(n,k)$ ning qiymati hisoblansin, har bir hisoblash jarayonida hosil bo'lgan

qiymatlar ikki o'lchovli massivda tasvirlansin, $n=20$ dan oshmagan qiymatida **Combin2** funksiyasining chaqiruvlar soni **Combin1** funksiyasining chaqiruvlar soni bilan taqqoslansin.

$$y_n - \frac{x}{y_n^{k-1}}$$

7. k tartibli ildizni hisoblaydigan ($y_0=1, y_{n+1} = y_n - \frac{x}{y_n^{k-1}}$) rekursiv haqiqiy tipli **RootK**(x, k, n) funksiyasi tasvirlansin. Bu erda y_n **RootK**(x, k, n) funksiyadagi fikserlangan x va k ni bildiradi. Funksiya parametrlari: $x(x>0)$ -haqiqiy son $k(k>1)$ va $n(n>0)$ -butun sonlar. **RootK** funksiyasi yordamida berilgan x soni uchun uning k -tartibli ildizini n ning 6 ta turli qiymatlarida hisoblansin.
8. 2 ta butun sonning umumiy bo'luvchisini Evklid algoritmi yordamida topadigan (**NOD**(a, b)=**NOD**($b, a \bmod b$), agar $b \neq 0$; **NOD**($a, 0$)= a) rekursiv butun tipli **NOD**(a, b) funksiyasi tasvirlansin. Agar **A, B, C, D** musbat butun sonlar berilgan bo'lsa **NOD** funksiyasidan foydalanib **NOD**(a, b), **NOD**(a, c), **NOD**(a, d) lar hisoblansin.
9. k sonining raqamlari yig'indisini hisoblaydigan butun tipli **DigitSum**(k) rekursiv funksiyasi tasvirlansin, raqamlar yig'indisini hisoblashda sikl operatorlaridan foydalanilmasin. Bu funksiya yordamida turli 5 ta sonning raqamlari yig'indisini hisoblash amalga oshirilsin.
10. Sikl operatoridan foydalanmasdan $n(1 \leq n \leq 10)$ ta elementga ega a massivdagi eng katta elementni topuvchi rekursiv butun tipli **MaxElem**(a, n) funksiyasi tasvirlansin. Ushbu funksiya yordamida a, b, c masivlardagi eng katta sonlar topilsin.